

Predicción de rotación de empleados

Datos, preparación, modelos, resultados y conclusiones

Proyecto formativo de Big Data, aprendizaje automático y visualización.

Uso exclusivamente educativo. Los datos y resultados no están validados para evaluar, seleccionar, retener ni gestionar empleados. No deben aplicarse a personas ni a decisiones laborales.

El proyecto desarrolla un flujo educativo para analizar y clasificar la variable de rotación Attrition (Sí/No) en un conjunto de ejemplo de Recursos Humanos. El recorrido incluye revisión de calidad, integración de ficheros, ingeniería de variables, entrenamiento de modelos y comunicación de resultados.

El propósito es demostrar técnicas de Big Data y aprendizaje automático en un entorno reproducible. El trabajo no evalúa a una plantilla real ni propone decisiones de selección, promoción, retención o gestión de personas.

Objetivos técnicos

- Preparar y unir dos ficheros tabulares por identificador de empleado.
- Crear una etiqueta binaria y variables derivadas con PySpark.
- Comparar tres clasificadores de scikit-learn con un protocolo común.
- Explorar tasas y coeficientes, y presentar resultados en un prototipo de Power BI.

2. Datos y calidad

El fichero principal `data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv` contiene 1.470 registros y 35 columnas del ejercicio de rotación. La etiqueta original registra 237 casos "Yes" y 1.233 casos "No", una prevalencia del 16,1 %. Entre las variables hay departamento, puesto, viajes, horas extra, salario, antigüedad, satisfacción y otras características del registro.

El nombre y el contenido son compatibles con el conjunto denominado IBM HR Analytics Employee Attrition & Performance. IBM publica un tutorial de clasificación de rotación que utiliza un ejercicio de esa temática. El tutorial no verifica la procedencia exacta ni la licencia de redistribución de la copia incluida en este repositorio. Referencia del tutorial de IBM.

El segundo fichero, `data/raw/encuesta_clima.csv`, sí tiene origen conocido: lo genera el propio proyecto con `scripts/generate_demo_survey.py`. Para cada `EmployeeNumber` asigna al azar cinco valores enteros entre 1 y 5 usando la semilla 42: implicación, satisfacción, equilibrio vida-trabajo, relación con la jefatura y satisfacción con el trabajo remoto. El generador no consulta `Attrition`. Por ello, estas filas no son respuestas reales ni mediciones de empleados y no contienen una señal de clima observada.

Controles disponibles

`scripts/check_data_quality.py` comprueba que la clave exista y no tenga nulos, detecta duplicados por `EmployeeNumber`, calcula nulos por columna y valida que `Attrition` solo contenga Yes/No. El límite predeterminado para nulos es 25 %. El informe se escribe en `/output/metrics/quality_report.json`. No se atribuye aquí un resultado PASS porque la ejecución Spark de este entorno no se realizó.

Los datos incluidos son de demostración. No introduzcas datos personales, confidenciales ni de empleados reales.

3. Arquitectura y preparación

`docker-compose.yml` declara tres servicios locales: Jupyter, Spark Master y PostgreSQL. Jupyter publica el puerto 8889 solo en 127.0.0.1 y crea un token temporal al arrancar. Spark Master publica 7077 y 8080 solo en la interfaz local. PostgreSQL publica 5442 en localhost y persiste su directorio en `./postgres`. Las credenciales de PostgreSQL son valores de demostración.

Compose no configura un Spark Worker. El ETL usa `local[*]` por defecto, de modo que procesa en el propio contenedor Jupyter y no distribuye el trabajo al Master. Jupyter monta scripts, datos, resultados y notebooks desde el repositorio.

Pasos que realiza el ETL

- 1 Lee ambos CSV con detección de tipos y elimina espacios exteriores en campos de texto.
- 2 Normaliza el nombre de la clave y convierte las cinco escalas de la encuesta a enteros; añade el prefijo `survey_`.
- 3 Elimina duplicados por `EmployeeNumber` en ambas entradas y conserva todas las filas del fichero principal mediante una unión izquierda.
- 4 Crea `attrition_label` a partir de `Attrition` y elimina la columna textual original para evitar fuga directa del objetivo.
- 5 Añade `overtime_flag`, `income_yearly` y `tenure_ratio`. Este último divide `YearsAtCompany` por `TotalWorkingYears` más un epsilon pequeño.
- 6 Guarda el resultado en Parquet comprimido con Snappy y escribe recuentos y columnas en `/output/metrics/etl_metrics.json`.

El Parquet resultante tiene 1.470 filas y 43 columnas tras integrar encuesta, etiqueta y variables derivadas.

4. Recorrido reproducible

Con Docker Engine y Docker Compose v2 instalados, el README documenta este orden de trabajo. Los comandos se ejecutan desde la raíz del repositorio.

1. Iniciar el entorno y obtener el token de Jupyter

```
docker compose up -d --build
docker compose logs jupyter
```

Abrir <http://localhost:8889/lab> e introducir el token temporal que aparece en los registros. El valor del token no se guarda en el repositorio.

2. Generar la encuesta sintética

```
docker compose run --rm jupyter python /scripts/generate_demo_survey.py \
  --input /data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv \
  --output /data/raw/encuesta_clima.csv --seed 42
```

3. Revisar entradas y ejecutar ETL

```
docker compose run --rm jupyter python /scripts/check_data_quality.py \
  --input1 /data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv \
  --input2 /data/raw/encuesta_clima.csv \
  --key EmployeeNumber --max-null-frac 0.25
```

```
docker compose run --rm jupyter python /scripts/etl_attrition.py \
  --input1 /data/raw/WA_Fn-UseC_-HR-Employee-Attrition.csv \
  --input2 /data/raw/encuesta_clima.csv \
  --outdir /data/processed/employee_attrition.parquet
```

4. Entrenar y abrir los notebooks

```
docker compose run --rm jupyter python /scripts/train_ml.py \
  --input /data/processed/employee_attrition.parquet --model all
```

Ejecutar en Jupyter, en orden: 01_EDA_Attrition, 02_Modelado_Baseline, 03_Modelado_DL, 05_Dashboard_KPIs y 99_Informe_Final. Para detener los servicios: `docker compose down`.

5. Análisis y modelos

Análisis exploratorio

El notebook 01_EDA_Attrition explora la estructura, distribución de la etiqueta y variables numéricas y categóricas. 05_Dashboard_KPIs calcula indicadores generales y tasas por departamento, puesto, horas extra, tramos de salario y antigüedad, y exporta coeficientes o importancias para el panel.

Preparación para clasificación

El entrenamiento excluye EmployeeNumber, EmployeeCount, StandardHours, Over18 y las columnas de etiqueta. El pipeline escala las variables numéricas con StandardScaler y codifica las categóricas con OneHotEncoder, que ignora categorías no vistas durante la transformación. Cada pipeline incluye su preprocesamiento dentro del ciclo de validación.

Clasificadores

- Regresión logística: solver saga, penalización L2, máximo de 3.000 iteraciones y ponderación balanceada de clases.
- Random Forest: 300 árboles, ponderación balanced_subsample y semilla fija 42.
- Perceptrón multicapa (MLP) de scikit-learn: capas ocultas 64 y 32, activación ReLU, máximo 150 iteraciones, parada temprana y semilla 42. El notebook se llama Modelado_DL, pero este MLP no usa un framework de aprendizaje profundo.

scripts/train_ml.py --model all calcula probabilidades fuera de muestra mediante validación cruzada estratificada de cinco particiones, genera las métricas, curvas y artefactos, y vuelve a ajustar cada pipeline con todos los datos para guardar el modelo final.

6. Resultados cuantitativos

El fichero output/metrics/model_compare.json contiene las métricas calculadas con el mismo esquema OOF para los tres clasificadores. Valores redondeados a tres decimales:

Modelo	ROC-AUC	PR-AUC	F1 ajustado	Umbral F1	Exactitud a 0,5	F1 a 0,5
Regresión logística	0,819	0,552	0,533	0,732	0,753	0,479
Bosque aleatorio	0,810	0,532	0,544	0,210	0,856	0,248
MLP	0,768	0,484	0,470	0,238	0,857	0,305

La regresión logística lidera ROC-AUC y PR-AUC, que describen la ordenación de casos y el rendimiento de precisión-recobrado. Random Forest logra el F1 optimizado más alto. El umbral de cada fila se selecciona para maximizar F1 en las mismas predicciones OOF evaluadas; esa reutilización introduce optimismo. Para estimar generalización habría que reservar datos independientes o ajustar el umbral dentro de un esquema anidado.

La etiqueta está desequilibrada: 16,1 % de positivos. Un clasificador que siempre predijera “No” alcanzaría aproximadamente 0,839 de exactitud, con F1 igual a cero para Attrition=Yes. Por eso la exactitud por sí sola puede ocultar un rendimiento pobre en la clase minoritaria. Random Forest y MLP tienen exactitud cercana a 0,86 a umbral 0,5, pero F1 de 0,248 y 0,305, respectivamente.

Los modelos guardados no son umbrales operativos. Las probabilidades no se calibraron y las métricas no se contrastaron en otro periodo u organización.

7. Lectura de las visualizaciones

Distribución de puntuaciones

El histograma reúne 1.470 predicciones OOF de la regresión logística en intervalos de 0,10. 773 puntuaciones (52,6 %) están por debajo de 0,30. Describe la salida del modelo en este conjunto; no representa probabilidades calibradas ni niveles de riesgo validados.

Tasa observada por departamento

Ventas registra 20,6 % (92 de 446), Recursos Humanos 19,0 % (12 de 63) e Investigación y Desarrollo 13,8 % (133 de 961). Se trata de proporciones observadas dentro de cada grupo. No explican por qué ocurre la rotación ni permiten inferir causas.

Tasa observada y horas extra

La tasa es 30,5 % con horas extra (127 de 416) y 10,4 % sin ellas (110 de 1.054). La diferencia es descriptiva; no demuestra que las horas extra causen las salidas.

Coeficientes de regresión logística

Los mayores coeficientes absolutos incluyen representante de ventas (+1,070), viajes frecuentes (+0,856), director/a de investigación (−1,121), ausencia de viajes (−0,850) y técnico de laboratorio (+0,820). El signo indica la dirección de asociación dentro del pipeline codificado. La magnitud depende de las categorías de referencia, el preprocesamiento y el conjunto; no es efecto causal ni recomendación laboral.

Power BI: el fichero PBIX es un prototipo. Las imágenes PNG en docs/powerbi son ilustraciones estáticas generadas para mostrar resúmenes de datos versionados. No son fotografías de la pantalla de Power BI y no certifican que el PBIX se haya abierto, que los filtros funcionen o que sus páginas coincidan con esos resúmenes.

8. Conclusiones, límites y siguientes pasos

Conclusiones del trabajo

- 1 El flujo demuestra cómo integrar un CSV de plantilla de ejemplo con una encuesta sintética, preparar variables y comparar modelos de clasificación.
- 2 La regresión logística ordena mejor los casos según ROC-AUC y PR-AUC. Random Forest presenta el F1 más alto solo tras ajustar el umbral sobre las predicciones de evaluación.
- 3 El MLP no supera a los otros modelos en las métricas de ordenación. La exactitud alta a umbral 0,5 debe leerse junto al desequilibrio de clases y los valores de F1.
- 4 Las diferencias por departamento, horas extra y coeficientes son patrones del conjunto de ejemplo. No demuestran causas ni se extrapolan a una empresa.

En conjunto, el trabajo acredita un ejercicio formativo de preparación de datos, clasificación y comunicación de resultados. No acredita una herramienta desplegada ni un modelo validado para predecir la rotación real de una organización.

Límites y trabajo futuro

- Ejecutar y registrar la cadena real Docker/Spark desde los CSV hasta las métricas. En esta revisión no estaban disponibles Docker Compose ni PySpark; las métricas se regeneraron con un Parquet local que siguió las transformaciones y nombres de columnas del ETL.
- Obtener y documentar las condiciones de redistribución del conjunto HR antes de distribuirlo por separado.
- Definir una evaluación independiente, temporal o anidada, calibrar probabilidades y medir incertidumbre.
- Revisar privacidad, sesgo y equidad antes de cualquier estudio con datos laborales reales.
- Abrir el PBIX en Power BI Desktop y revisar páginas, filtros, relaciones y correspondencia entre visuales y datos exportados.

No se deben usar estas métricas para evaluar personas, asignar recursos de RR. HH. ni tomar decisiones laborales.

Apéndice. Salidas y persistencia

- `/output/metrics/model_compare.json`: comparación común OOF.
- `/output/models/`: pipelines ajustados para cada modelo.
- `/output/plots/`: curvas ROC y precisión-recobrado.
- `/output/bi/predictions_logreg.csv` y `feature_effects.csv`: exportaciones descriptivas para prototipos.
- `/output/metrics/etl_metrics.json` y `quality_report.json`: recuentos de ETL y controles, si se ejecutan.

`scripts/load_features_to_pg.py` crea y carga la tabla `employees_features` en PostgreSQL. `scripts/persist_results.py` guarda métricas y predicciones, pero espera que las tablas `metrics` y `predictions` ya existan; el repositorio no incluye su esquema SQL. Para conectar con el Compose de este proyecto hay que pasar `host postgres`, `base mlops` y el usuario/contraseña de demostración ml.

Las dependencias de aprendizaje automático usan rangos de versiones y no un fichero lock. Una reconstrucción idéntica requiere registrar y fijar el entorno de ejecución.

Proyecto educativo de Big Data, integración de datos, aprendizaje automático y visualización.